

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
 - o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
 - o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
 - o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
 - o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output:

1. **Python program that uses the hashlib library to create the hash of a given string:**

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()

    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))

    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()

    # Return the hash string
```

```
return hash_string
```

Output:

Enter a string: Jai Udasi

Name: Jai Udasi

Input String: Jai Udasi

Hash: a7a0290d8ad8c190a2bb9c2dda00f17510b375f9bacd9069ac0b69b67a78c617

2. Program to generate required target hash with input string and nonce:

```
import hashlib
```

```
# Get user input
```

```
input_string = input("Enter a string: ")
```

```
nonce = input("Enter the nonce: ")
```

```
# Concatenate the string and nonce
```

```
hash_string = input_string + nonce
```

```
# Calculate the hash using SHA-256
```

```
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
```

```
hash_code = hash_object.hexdigest()
```

```
# Print the hash code
```

```
print("Name: Jai Udasi")
```

```
print("Input String:", input_string)
```

```
print("Nonce:", nonce)
```

```
print("Hash Code:", hash_code)
```

Output:

Enter a string: Jai Udasi

Enter the nonce: 8232

Name: Jai Udasi

Input String: Jai Udasi

Nonce: 8232

Hash Code:

0000142ae4ecf85d9ba33b6c9406474cb3fbe9eff4cf66659bd26c028bf98ef1

3. Python code for Solving Puzzle for leading zeros with expected nonce and given string:

```
import hashlib
```

```
def find_nonce(input_string, num_zeros):
```

```
    nonce = 0
```

```
    hash_prefix = '0' * num_zeros
```

```
    while True:
```

```
        # Concatenate input string and nonce
```

```
        hash_string = input_string + str(nonce)
```

```
        # Calculate SHA-256 hash
```

```
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
```

```
        hash_code = hash_object.hexdigest()
```

```
        # Check whether hash starts with required zeros
```

```
        if hash_code.startswith(hash_prefix):
```

```
            print("\nHash Found:", hash_code)
```

```
            return nonce
```

```
        nonce += 1
```

```
# Get user input
```

```
input_string = input("Enter a string: ")
```

```
num_zeros = int(input("Enter the number of leading zeros required: "))

# Find the nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Display result
print("\nName: Jai Udasi")
print("Input String:", input_string)
print("Leading Zeros Required:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

Output:

Enter a string: Jai Udasi

Enter the number of leading zeros required: 4

Hash Found:

0000142ae4ecf85d9ba33b6c9406474cb3fbe9eff4cf66659bd26c028bf98ef1

Name: Jai Udasi

Input String: Jai Udasi

Leading Zeros Required: 4

Expected Nonce: 8232

4. Generating Merkle Tree for given set of Transactions:

```
import hashlib
```

FUNCTION 1: Generate SHA-256 Hash

```
def hash_data(data):  
    return hashlib.sha256(data.encode("utf-8")).hexdigest()
```

FUNCTION 2: Shorten Hash for Tree Display

```
def short_hash(hash_value):  
    return hash_value[:10] + "..."
```

FUNCTION 3: Print Intermediate Hashes

```
def print_hash_levels(levels):  
  
    print("\n" + "=" * 90)  
    print("          INTERMEDIATE MERKLE TREE HASHES")  
    print("=" * 90)  
  
    for level_no, level in enumerate(levels):  
  
        if level_no == 0:  
            print("\nLEVEL 0 - TRANSACTION HASHES")  
        else:  
            print(f"\nLEVEL {level_no}")  
  
        for i, node in enumerate(level):  
  
            print(f" H{i + 1}: {node}")
```

FUNCTION 4: Build Merkle Tree

```
def build_merkle_tree(transactions):
```

```
# Hash every transaction first
current_level = []

for transaction in transactions:
    current_level.append(hash_data(transaction))

# Store all levels of the tree
levels = [current_level.copy()]

while len(current_level) > 1:

    # If number of hashes is odd,
    # duplicate the last hash
    if len(current_level) % 2 != 0:
        current_level.append(current_level[-1])

    next_level = []

    # Combine hashes in pairs
    for i in range(0, len(current_level), 2):

        combined = current_level[i] + current_level[i + 1]

        parent_hash = hash_data(combined)

        next_level.append(parent_hash)
    current_level = next_level

    levels.append(current_level.copy())
```

```
return current_level[0], levels
```

```
# FUNCTION 5: Print ASCII MERKLE TREE
```

```
def print_merkle_tree(transactions, levels):
```

```
    print("\n\n" + "=" * 90)
```

```
    print("                MERKLE TREE")
```

```
    print("=" * 90)
```

```
    # -----
```

```
    # Print levels from ROOT to TRANSACTIONS
```

```
    # -----
```

```
    root_level = levels[-1]
```

```
    print("\n                |_____|")
    print("                |  MERKLE ROOT  |")
    print(f"                | {short_hash(root_level[0]):^20} |")
    print("                |_____|")
```

```
    # Work backwards from second-last level
```

```
    for level_index in range(len(levels) - 2, -1, -1):
```

```
        current_level = levels[level_index]
```

```
        print("                |")
```

```
    # Print connecting line
```

```

print("          |")

# Display nodes
if len(current_level) == 1:

    print(
        f"          | "
        f"[{short_hash(current_level[0])}]"
        f" | "
    )

else:

    # Create pairs for display
    for i in range(0, len(current_level), 2):

        left = short_hash(current_level[i])

        if i + 1 < len(current_level):
            right = short_hash(current_level[i + 1])
        else:
            right = left

        print(
            f"          [{left}]   [{right}]"
        )

# Separator between levels
if level_index > 0:
    print("          |")

```



```
# Print Transactions at Bottom
```

```
print("\n")
```

```
print("-" * 90)
```

```
print("          TRANSACTIONS")
```

```
print("-" * 90)
```

```
for i, transaction in enumerate(transactions):
```

```
    print(
```

```
        f"T {i + 1} -> {transaction}"
```

```
    )
```

```
print("-" * 90)
```

```
# MAIN PROGRAM
```

```
print("\n" + "=" * 90)
```

```
print("          MERKLE TREE GENERATOR")
```

```
print("          Jai Udasi")
```

```
print("=" * 90)
```

```
# USER INPUT
```

```
while True:
```

```
    try:
```

```
        number = int(
```

```
            input("\nEnter number of transactions: ")
```

```
)
```

```
if number > 0:
```

```
    break
```

```
print("Please enter a number greater than 0.")
```

```
except ValueError:
```

```
    print("Please enter a valid integer.")
```

```
transactions = []
```

```
print("\nEnter your transactions:")
```

```
for i in range(number):
```

```
    transaction = input(f"T {i + 1}: ")
```

```
    transactions.append(transaction)
```

```
# DISPLAY TRANSACTION INFORMATION
```

```
print("\n" + "=" * 90)
```

```
print("          INPUT TRANSACTIONS")
```

```
print("=" * 90)
```

```
for i, transaction in enumerate(transactions):
```

```
print(f"T{i + 1}: {transaction}")

# HASH EACH TRANSACTION
print("\n" + "=" * 90)
print("          TRANSACTION HASHES")
print("=" * 90)

for i, transaction in enumerate(transactions):

    transaction_hash = hash_data(transaction)

    print(f"\nT{i + 1}: {transaction}")
    print(f"Hash: {transaction_hash}")

# BUILD MERKLE TREE
merkle_root, levels = build_merkle_tree(transactions)

# PRINT INTERMEDIATE HASHES
print_hash_levels(levels)

# PRINT ASCII MERKLE TREE
print_merkle_tree(transactions, levels)

# FINAL RESULT
print("\n" + "=" * 90)
print("          FINAL RESULT")
print("=" * 90)

print("Name          :", "Jai Udasi")
```

```
print("Number of Transactions:", number)
print("Merkle Root      :", merkle_root)

print("=" * 90)
```

Output:

```
=====
MERKLE TREE GENERATOR
```

```
      Jai Udasi
=====
```

```
Enter number of transactions: 5
```

```
Enter your transactions:
```

```
T1: Alice -> Bob : $200
```

```
T2: Bob -> Dave : $500
```

```
T3: Dave -> Eve : $100
```

```
T4: Eve -> Alice : $300
```

```
T5: Roo -> Bob : $50
```

```
=====
INPUT TRANSACTIONS
```

```
=====T1:
Alice -> Bob : $200
```

```
T2: Bob -> Dave : $500
```

```
T3: Dave -> Eve : $100
```

```
T4: Eve -> Alice : $300
```

```
T5: Roo -> Bob : $50
```

=====

TRANSACTION HASHES

=====

T1: Alice -> Bob : \$200

Hash: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760

T2: Bob -> Dave : \$500

Hash: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46

T3: Dave -> Eve : \$100

Hash: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da

T4: Eve -> Alice : \$300

Hash: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a

T5: Roo -> Bob : \$50

Hash: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

=====

INTERMEDIATE MERKLE TREE HASHES

=====

LEVEL 0 - TRANSACTION HASHES

H1: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760

H2: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46

H3: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da

H4: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a

H5: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

LEVEL 1

H1: 471608d3d6f2a642ff3a84dfaeaeabb1c271a9aed5b37a82d61c4784558e80e9

H2: a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

H3: 314ca746b3daf63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

LEVEL 2

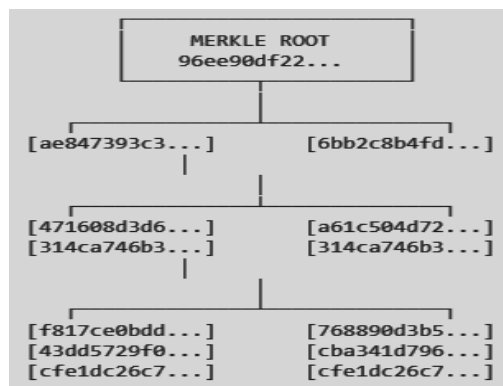
H1: ae847393c3dff60e1d95cfdabb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

H2: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

LEVEL 3

H1: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

MERKLE TREE



TRANSACTIONS

T1 -> Alice -> Bob : \$200

T2 -> Bob -> Dave : \$500

T3 -> Dave -> Eve : \$100

T4 -> Eve -> Alice : \$300

T5 -> Roo -> Bob : \$50

=====

FINAL RESULT

=====Name

me : Jai Udasi

Number of Transactions: 5

Merkle Root:

96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

=====

Conclusion

The experiment provided a practical understanding of fundamental cryptographic techniques and their role in Blockchain technology. The following concepts were successfully implemented and studied:

- **SHA-256 Hashing** was used to generate a fixed-length cryptographic hash for input data, demonstrating how hashing helps maintain data integrity.
- **Nonce and Target Hashing** demonstrated the basic working of Proof-of-Work, where a suitable nonce is searched for to generate a hash satisfying a specified condition, such as a required number of leading zeros.
- **Proof-of-Work Puzzle Solving** showed how computational effort is required to find a valid nonce and how increasing the difficulty of the hash condition makes the puzzle more challenging.
- **Merkle Tree Construction** demonstrated how multiple transactions can be represented efficiently using a single Merkle Root hash. Changes to any transaction affect the resulting hashes and can therefore be detected through the Merkle Root.

Overall, the experiment demonstrated how cryptographic hashing, nonce-based puzzles, Proof-of-Work, and Merkle Trees contribute to the **security, integrity, immutability, and efficient verification of data in Blockchain systems**.